

Autenticação & Arquitetura

GUIA DE REVISÃO — PADRÃO SISP

GAMEDB

Este guia resume o caminho de uma requisição pelas camadas e o fluxo completo de login/logout/permisões, exatamente como implementado no projeto. Use como reforço rápido antes da prova.

1. AS CAMADAS E SUAS RESPONSABILIDADES

Regra mestra: **cada camada faz UMA coisa**. O que define onde algo mora é a pergunta "que tipo de responsabilidade é essa?".

CAMADA	RESPONSABILIDADE	NUNCA FAZ
Controller THIN	Coordena: recebe request, chama DTO/Service, devolve resposta.	Query, lógica de negócio, validação inline, <code>map()</code> /array manual.
DTO	Valida formato do campo (obrigatório, e-mail válido, confirmação de senha).	Unicidade, regra de negócio, query.
Service	Regra de negócio + <code>DB::transaction</code> . Orquestra Model + Repository.	Query direta, montar array formatado p/ apresentação.
Repository	TODA query Eloquent + validação de unicidade + persistir (<code>save</code>).	Regra de negócio, formatação.
Model	Monta a entidade (<code>criar()</code> / <code>alterar()</code>), casts, relações.	Salvar a si mesmo, query.
Resource	Transforma entidade → resposta (JSON). Formata datas, esconde senha.	Lógica, decisão de negócio.

Caminho de uma request (ida)

```
Rota → Middleware (auth → permissao) → Controller (thin)
      → DTO (valida formato) → Service (regra + transaction)
      → Repository (query + unicidade) → Model → Banco
Volta: Model → Resource (formata) → JSON
```

Onde vive cada validação: Formato → **DTO** · Unicidade → **Repository** · Regra de negócio → **Service** · Cálculo de domínio → **Model**.

2. INJEÇÃO DE DEPENDÊNCIA & INTERFACES

Interface = contrato (só assinaturas, sem corpo). A classe `implements` a interface e promete cumprir.

O consumidor faz type-hint da **interface**, nunca da classe concreta:

```
public function __construct(
    protected IUserarioService $usuario_service,
) {}
```

O **binding** no `AppServiceProvider` liga interface → implementação:

```
$this->app->bind(
    IUserarioService::class,
    UsuarioService::class,
);
```

O **Service Container** resolve tudo em cascata: pede a interface, ele monta a implementação e todas as dependências dela. Você nunca escreve `new`.

Ao adicionar um método: coloque a assinatura na **interface** E a implementação na **classe**. Esquecer a interface = o Controller não "enxerga" o método.

Fluxo de Autenticação

LOGIN · SESSÃO · LOGOUT

GAMEDB

3. LOGIN — PASSO A PASSO

- 1 **Form + jQuery** — submit via AJAX. Headers obrigatórios: `X-CSRF-TOKEN`, `X-Requested-With: XMLHttpRequest`, `Accept: application/json`. Body em JSON.
- 2 **Controller.autenticar** — monta o DTO e chama o Service. Continua thin.
- 3 **UsuarioLoginDTO** — `validarLogin()` checa só **formato** (e-mail válido, senha presente). Nunca valida `min:6` no login — daria pista para ataque.
- 4 **Service.autenticar** — `Auth::attempt($dto->credenciais(), $dto->lembrar)`. Se falhar, `throw_unless(..., 'Credenciais inválidas.')` — mensagem sempre genérica.
- 5 **Controller** — após sucesso: `$request->session()->regenerate()` (previne session fixation) e devolve `redirect` no JSON.
- 6 **jQuery success** — `window.location.href = response.redirect` (redirect vem do servidor, não do front).

Código-chave do Service

```
public function autenticar(UsuarioLoginDTO $dados): Usuario
{
    throw_unless(
        Auth::attempt($dados->credenciais(), $dados->lembrar),
        new \Exception('Credenciais inválidas.')
    );

    /** @var Usuario $usuario */
    $usuario = Auth::user();
    return $usuario;
}
```

Auth::attempt(\$cred, \$lembrar)

Recebe (array, bool). Busca pelo e-mail (chaves do array = colunas), compara o hash da senha, e loga na sessão. Retorna bool.

Onde mora o quê

SERVICE "as credenciais batem?" · **CONTROLLER** "blindar a sessão" (`regenerate`). O Service não conhece HTTP/sessão.

4. "LEMBRAR DE MIM" & LOGOUT

Remember me

Quando `$lembrar = true`, o Auth gera um token, salva em `remember_token` (`sg_usuarios`) e manda um cookie persistente criptografado.

- Não preenche campos — **pula o login** quando a sessão expira.
- Só dispara ao acessar rota protegida com sessão vazia.
- No front: `$('#lembrar').is(':checked')` (não `.val()`).

Logout

```
// Service
Auth::logout();
Cache::forget('permissoes_usuario_'. $id);
// Controller
$request->session()->invalidate();
$request->session()->regenerateToken();
```

Rota **POST** (nunca GET — evita logout via `<img>`). Pega `Auth::id()` antes do `logout()`.

Permissões & Regras de Ouro

MIDDLEWARE · HELPER · CHEAT SHEET

GAMEDB

5. SISTEMA DE PERMISSÕES

Regra mestra: "**permissão**" = **nome de rota**. Um usuário pode fazer X se algum perfil dele tem a linha `str_name = 'modulo.entidade.acao'`. Nunca `if ($user->is_admin)`.

Usuario → perfil_id → Perfil ↔ perfil_permissao ↔ Permissao

Helper (Blade)

```
if (!Auth::check()) return false;

$permissoes = Cache::remember(
    'permissoes_usuario_' . Auth::id(),
    3600, // 1h TTL
    fn() => $usuario->perfil->permissoes
        ->pluck('str_name')->toArray() ?? []
);
return in_array($str_rota, $permissoes);
```

Uso: `@if (possuiPermissao('...'))` esconde o botão.

Middleware (rota)

```
$rota = $request->route()->getName();

if (!possuiPermissao($rota)) {
    abort(403);
}
return $next($request);
```

Genérico: o nome da rota == nome da permissão. **Mesmo helper** que o Blade usa (DRY).

Rota: `->middleware(['auth', 'permissao'])` — **auth antes** de permissao.

Ownership vs Permissão: Permissão de rota = "pode usar essa ação em geral" (middleware). Ownership = "pode mexer NESTE registro" (ex.: admin ou dono) → regra de negócio no **Service**, via `private function validar*()`.

6. PIPELINE DE MIDDLEWARE (ROTA PROTEGIDA)

- 1 **auth** — tem usuário logado? Se não → redirect p/ login (ou 401 JSON se for AJAX).
- 2 **permissao** — `possuiPermissao(nome_da_rota)` ? Se não → `abort(403)`.
- 3 **Controller** — request autorizada, executa a ação.

7. REGRAS DE OURO SISP (CHEAT SHEET)

REGRA	DETALHE
<code>declare(strict_types=1)</code>	Em Service, Repository, Model, DTO. Nunca em Controller (HTTP traz string).
Named arguments	Em toda chamada — inclusive helpers e funções nativas: <code>view(view: ...)</code> , <code>in_array(needle: ...)</code> .
<code>throw_if</code> / <code>throw_unless</code>	No lugar de <code>if/throw</code> . Repository valida unicidade; Service valida regra.
Return types explícitos	Em todo método, inclusive <code>private</code> e closures.
Seeder = Rota = Controller	O nome da permissão é o mesmo nos 3 lugares. camelCase: <code>modulo.entidade.acao</code> .
GET lê, POST escreve	<code>listar/buscar/visualizar</code> = GET · <code>incluir/alterar/remover</code> = POST.
Controller injeta Service	Sempre a interface, nunca Repository direto, nunca classe concreta.
<code>Resource::criar()</code>	Ponto único de entrada: aceita Model, Collection ou Paginator.